Modul Pembelajaran CodeIgniter 4:
Membuat Sistem Informasi Toko

Modul ini disusun khusus untuk belajar membangun Sistem Informasi Toko sederhana (Kasir/Point of Sales) menggunakan framework CodeIgniter 4.

Tujuan Pembelajaran:

- Memahami konsep MVC (Model-View-Controller) pada CodeIgniter 4.
- Mampu melakukan instalasi dan konfigurasi awal CodeIgniter 4.
- Mampu membuat dan mengelola database MySQL.
- Mampu melakukan operasi CRUD (Create, Read, Update, Delete) pada tabel data.
- Membangun aplikasi Point of Sales (POS) dasar.

Bab 1: Persiapan dan Instalasi

1.1 Persiapan Sistem

Sebelum memulai, pastikan perangkat lunak berikut sudah terpasang di komputer Anda:

1. **XAMPP / Laragon**: Untuk web server (Apache) dan database server (MySQL/MariaDB). Pastikan versi PHP minimal 7.4 (disarankan PHP 8.1+ untuk CI4 terbaru).
2. **Composer**: Dependency manager untuk PHP. Unduh di `getcomposer.org`.
3. **Visual Studio Code (VS Code)**: Text editor pilihan.
4. **Web Browser**: Chrome, Firefox, atau Edge.

1.2 Instalasi CodeIgniter 4

Kita akan menggunakan Composer untuk menginstal CodeIgniter 4.

1. Buka **Terminal** atau **Command Prompt**.
2. Masuk ke direktori web server Anda (contoh: `htdocs` pada XAMPP).

```
cd C:\xampp\htdocs
```

4. Jalankan perintah berikut untuk menginstal CI4 dengan nama folder `sisfo_toko`:

```
composer create-project codeigniter4/appstarter sisfo_toko
```

6. Tunggu hingga proses unduhan selesai.
7. Masuk ke folder proyek:

```
cd sisfo_toko
```

9. Jalankan server pengembangan CI4 untuk memastikan instalasi berhasil:

```
php spark serve
```

11. Buka browser dan akses `http://localhost:8080`. Jika muncul halaman "Welcome to CodeIgniter 4", instalasi berhasil. Tekan `Ctrl + C` di terminal untuk mematikan server.

1.3 Konfigurasi Awal

1. Buka folder `sisfo_toko` di VS Code.
2. Cari file bernama `env` (tanpa titik di depan). Ubah nama file tersebut menjadi `.env`.
3. Buka file `.env`, cari baris berikut, hilangkan tanda pagar (`#`) dan ubah menjadi `development` agar pesan error tampil saat proses pembuatan aplikasi:

```
CI_ENVIRONMENT = development
```

5. Konfigurasi URL utama (Base URL). Cari dan ubah baris berikut:

```
app.baseURL = 'http://localhost:8080/'
```

Bab 2: Desain dan Konfigurasi Database

2.1 Membuat Database MySQL

1. Pastikan Apache dan MySQL di XAMPP sudah berjalan.
2. Buka phpMyAdmin di browser (`http://localhost/phpmyadmin`).
3. Buat database baru dengan nama `db_toko`.

2.2 Konfigurasi Database di CI4

1. Buka kembali file `.env` di VS Code.
2. Cari bagian `database.default`, hilangkan tanda pagar (`#`), dan sesuaikan dengan konfigurasi MySQL Anda:

```
database.default.hostname = localhost
database.default.database = db_toko
database.default.username = root
database.default.password =
database.default.DBDriver = MySQLi
database.default.DBPrefix =
database.default.port = 3306
```

2.3 Membuat Tabel Menggunakan Migration (Opsional namun Disarankan)

CodeIgniter memiliki fitur Migration untuk membuat struktur tabel dari kode.

1. Buka terminal di VS Code (Ctrl + `).
2. Jalankan perintah berikut untuk membuat file migrasi:

```
php spark make:migration Produk
```

4. Buka file migrasi yang baru terbuat di `app/Database/Migrations/`.
5. Ubah isi method `up()` dan `down()` menjadi seperti ini:

```
<?php

```

```
8. namespace App\Database\Migrations;
9.
10. use CodeIgniter\Database\Migration;
11.
12. class Produk extends Migration
13. {
14.     public function up()
15.     {
16.         $this->forge->addField([
17.             'id_produk' => [
18.                 'type'           => 'INT',
19.                 'constraint'     => 11,
20.                 'unsigned'       => true,
21.                 'auto_increment' => true,
22.             ],
23.             'kode_produk' => [
24.                 'type'       => 'VARCHAR',
25.                 'constraint' => '50',
26.             ],
27.             'nama_produk' => [
28.                 'type'       => 'VARCHAR',
29.                 'constraint' => '100',
30.             ],
31.             'harga' => [
32.                 'type'       => 'INT',
33.                 'constraint' => 11,
34.             ],
35.             'stok' => [
36.                 'type'       => 'INT',
37.                 'constraint' => 11,
38.                 'default'    => 0,
39.             ],
40.             'created_at' => [
41.                 'type' => 'DATETIME',
42.                 'null' => true,
43.             ],
44.             'updated_at' => [
45.                 'type' => 'DATETIME',
46.                 'null' => true,
47.             ],
48.         ]);
```

```
        $this->forge->addKey('id_produk', true);
        $this->forge->createTable('produk');
    }

    public function down()
    {
        $this->forge->dropTable('produk');
    }
}
```

58. Jalankan migrasi di terminal:

```
php spark migrate
```

Cek phpMyAdmin, tabel `produk` seharusnya sudah terbentuk.

Bab 3: Membuat Model (Model)

Model bertugas mengelola interaksi dengan database.

1. Jalankan perintah di terminal:

```
php spark make:model ProdukModel
```

3. Buka file `app/Models/ProdukModel.php`. Sesuaikan isinya:

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class ProdukModel extends Model
{
    protected $table            = 'produk';
    protected $primaryKey       = 'id_produk';
    protected $useAutoIncrement = true;
    protected $returnType       = 'array';
    protected $useSoftDeletes   = false;

    // Field yang diizinkan untuk diisi (mass assignment)
    protected $allowedFields    = ['kode_produk',
  'nama_produk', 'harga', 'stok'];

    // Aktifkan timestamp otomatis
    protected $useTimestamps = true;
```

```
    protected $createdField  = 'created_at';
    protected $updatedField  = 'updated_at';
}
```

Bab 4: Tampilan (View) dan Bootstrap

Kita akan menggunakan Bootstrap 5 agar tampilan aplikasi rapi dan responsif.

4.1 Membuat Template Dasar

1. Buat folder baru bernama `layout` di dalam `app/Views/`.
2. Buat file `template.php` di dalam `app/Views/layout/`.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?= $title ?? 'Sistem Informasi Toko' ?></title>
    <!-- Bootstrap CSS CDN -->
     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <!-- Navbar -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
        <div class="container">
            <a class="navbar-brand" href="/">Toko SMK</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-toggle="collapse" data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav">
                    <li class="nav-item">
                        <a class="nav-link" href="/produk">Data Produk</a>
                    </li>
                </ul>
            </div>
```

```
        </div>
    </nav>

    <!-- Area Konten Utama -->
    <div class="container">
        <?= $this->renderSection('content') ?>
    </div>

    <!-- Bootstrap JS CDN -->
    <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/b
    ootstrap.bundle.min.js"></script>
</body>
</html>
```

Bab 5: Controller dan Operasi CRUD (Produk)

5.1 Membuat Controller Produk

1. Jalankan perintah di terminal:

```
php spark make:controller Produk
```

3. Buka file `app/Controllers/Produk.php`.

5.2 Read (Menampilkan Data)

Tambahkan kode berikut pada `Produk.php`:

```
<?php

namespace App\Controllers;

use App\Controllers\BaseController;
use App\Models\ProdukModel;

class Produk extends BaseController
{
    protected $produkModel;

    public function __construct()
    {
        // Instansiasi model agar bisa digunakan di semua method
        $this->produkModel = new ProdukModel();
    }

    public function index()
```

```
    {
        $data = [
            'title' => 'Data Produk',
            'produk' => $this->produkModel->findAll() //
Mengambil semua data
        ];

        return view('produk/index', $data);
    }
}
```

Buat view-nya. Buat folder `produk` di `app/Views/`. Lalu buat file `index.php` di dalamnya:

```
<?= $this->extend('layout/template') ?>

<?= $this->section('content') ?>
<div class="row">
    <div class="col">
        <h2 class="mb-3">Daftar Produk</h2>

        <?php if(session()->getFlashdata('pesan')) : ?>
            <div class="alert alert-success" role="alert">
                <?= session()->getFlashdata('pesan'); ?>
            </div>
        <?php endif; ?>

        <a href="/produk/create" class="btn btn-primary mb-
3">Tambah Data Produk</a>

        <table class="table table-bordered table-striped">
            <thead>
                <tr>
                    <th>No</th>
                    <th>Kode Produk</th>
                    <th>Nama Produk</th>
                    <th>Harga</th>
                    <th>Stok</th>
                    <th>Aksi</th>
                </tr>
            </thead>
            <tbody>
                <?php $i = 1; foreach($produk as $p) : ?>
```

```
                <tr>
                    <td><?= $i++; ?></td>
                    <td><?= $p['kode_produk']; ?></td>
                    <td><?= $p['nama_produk']; ?></td>
                    <td>Rp <?= number_format($p['harga'], 0,
',', '.'); ?></td>
                    <td><?= $p['stok']; ?></td>
                    <td>
                        <a href="/produk/edit/<?=
$p['id_produk']; ?>" class="btn btn-warning btn-sm">Edit</a>

                        <form action="/produk/<?=
$p['id_produk']; ?>" method="post" class="d-inline">
                            <?= csrf_field(); ?>
                            <input type="hidden" name="_method"
value="DELETE">
                            <button type="submit" class="btn
btn-danger btn-sm" onclick="return confirm('Apakah Anda
yakin?');">Hapus</button>
                        </form>
                    </td>
                </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    </div>
</div>
<?= $this->endSection() ?>
```

5.3 Create (Menambah Data)

Tambahkan method `create` dan `save` di `Controllers/Produk.php`:

```
    public function create()
    {
        $data = [
            'title' => 'Form Tambah Produk'
        ];
        return view('produk/create', $data);
    }

    public function save()
    {
```

```
        // Validasi sederhana bisa ditambahkan di sini

        $this->produkModel->save([
            'kode_produk' => $this->request-
>getVar('kode_produk'),
            'nama_produk' => $this->request-
>getVar('nama_produk'),
            'harga'       => $this->request->getVar('harga'),
            'stok'        => $this->request->getVar('stok')
        ]);

        session()->setFlashdata('pesan', 'Data berhasil
ditambahkan.');
        return redirect()->to('/produk');
    }
```

Buat file app/Views/produk/create.php:

```
<?= $this->extend('layout/template') ?>

<?= $this->section('content') ?>
<div class="row">
    <div class="col-8">
        <h2 class="my-3">Form Tambah Produk</h2>
        <form action="/produk/save" method="post">
            <?= csrf_field(); ?>
            <div class="mb-3">
                <label for="kode_produk" class="form-label">Kode
Produk</label>
                <input type="text" class="form-control"
id="kode_produk" name="kode_produk" required autofocus>
            </div>
            <div class="mb-3">
                <label for="nama_produk" class="form-label">Nama
Produk</label>
                <input type="text" class="form-control"
id="nama_produk" name="nama_produk" required>
            </div>
            <div class="mb-3">
                <label for="harga" class="form-
label">Harga</label>
```

```
                <input type="number" class="form-control"
id="harga" name="harga" required>
            </div>
            <div class="mb-3">
                <label for="stok" class="form-
label">Stok</label>
                <input type="number" class="form-control"
id="stok" name="stok" required>
            </div>
            <button type="submit" class="btn btn-
primary">Simpan</button>
            <a href="/produk" class="btn btn-
secondary">Kembali</a>
        </form>
    </div>
</div>
<?= $this->endSection() ?>
```

5.4 Update (Mengubah Data)

Tambahkan method `edit` dan `update` di `Controllers/Produk.php`:

```
    public function edit($id)
    {
        $data = [
            'title' => 'Form Ubah Data Produk',
            'produk' => $this->produkModel->find($id)
        ];
        return view('produk/edit', $data);
    }

    public function update($id)
    {
        $this->produkModel->save([
            'id_produk'   => $id,
            'kode_produk' => $this->request-
>getVar('kode_produk'),
            'nama_produk' => $this->request-
>getVar('nama_produk'),
            'harga'       => $this->request->getVar('harga'),
            'stok'        => $this->request->getVar('stok')
        ]);
```

```
        session()->setFlashdata('pesan', 'Data berhasil
diubah.');
        return redirect()->to('/produk');
    }
```

Buat file app/Views/produk/edit.php:

```
<?= $this->extend('layout/template') ?>

<?= $this->section('content') ?>
<div class="row">
    <div class="col-8">
        <h2 class="my-3">Form Ubah Data Produk</h2>
        <form action="/produk/update/<?= $produk['id_produk'];
?>" method="post">
            <?= csrf_field(); ?>
            <div class="mb-3">
                <label for="kode_produk" class="form-label">Kode
Produk</label>
                <input type="text" class="form-control"
id="kode_produk" name="kode_produk" value="<?=
$produk['kode_produk']; ?>" required>
            </div>
            <div class="mb-3">
                <label for="nama_produk" class="form-label">Nama
Produk</label>
                <input type="text" class="form-control"
id="nama_produk" name="nama_produk" value="<?=
$produk['nama_produk']; ?>" required>
            </div>
            <div class="mb-3">
                <label for="harga" class="form-
label">Harga</label>
                <input type="number" class="form-control"
id="harga" name="harga" value="<?= $produk['harga']; ?>"
required>
            </div>
            <div class="mb-3">
                <label for="stok" class="form-
label">Stok</label>
                <input type="number" class="form-control"
id="stok" name="stok" value="<?= $produk['stok']; ?>" required>
```

```
            </div>
            <button type="submit" class="btn btn-primary">Ubah
Data</button>
            <a href="/produk" class="btn btn-
secondary">Kembali</a>
        </form>
    </div>
</div>
<?= $this->endSection() ?>
```

5.5 Delete (Menghapus Data)

Tambahkan method `delete` di `Controllers/Produk.php`:

```
    public function delete($id)
    {
        $this->produkModel->delete($id);
        session()->setFlashdata('pesan', 'Data berhasil
dihapus.');
        return redirect()->to('/produk');
    }
```

5.6 Konfigurasi Routes

Agar route untuk delete berfungsi dengan baik (menggunakan method DELETE untuk keamanan), buka file `app/Config/Routes.php` dan sesuaikan:

```
use CodeIgniter\Router\RouteCollection;

/**
 * @var RouteCollection $routes
 */
$routes->get('/', 'Home::index');

// Routes untuk Produk
$routes->get('/produk', 'Produk::index');
$routes->get('/produk/create', 'Produk::create');
$routes->post('/produk/save', 'Produk::save');
$routes->get('/produk/edit/(:num)', 'Produk::edit/$1');
$routes->post('/produk/update/(:num)', 'Produk::update/$1');
$routes->delete('/produk/(:num)', 'Produk::delete/$1');
```

Bab 6: Menjalankan Aplikasi

1. Buka terminal, jalankan kembali server lokal:
2. `php spark serve`

3. Buka browser: `http://localhost:8080/produk`.
4. Cobalah untuk menambah, mengubah, dan menghapus data produk.

Selamat! Anda telah berhasil membuat Sistem Informasi Toko dasar dengan operasi CRUD lengkap menggunakan CodeIgniter 4.

Tugas Lanjutan (Tantangan untuk Siswa):

1. Tambahkan validasi form (misal: kode produk tidak boleh sama, field tidak boleh kosong).
2. Buat tabel dan fitur untuk "Transaksi Penjualan".
3. Tambahkan fitur pencarian produk.

Lanjutan Modul CodeIgniter 4:
Menyempurnakan Aplikasi (Dashboard & Laporan)

Pada modul sebelumnya, kita telah berhasil membuat fitur CRUD (Create, Read, Update, Delete) untuk data Produk. Sekarang, kita akan menyempurnakan Sistem Informasi Toko ini dengan menambahkan **Dashboard** sebagai pusat informasi (Analytics) dan fitur **Laporan** untuk mencetak data.

Bab 8: Membuat Dashboard Utama

Dashboard berfungsi untuk menampilkan ringkasan data secara cepat, seperti total produk, jumlah stok keseluruhan, dan estimasi nilai aset toko.

8.1 Membuat Controller Dashboard

1. Buka terminal di VS Code, pastikan Anda berada di folder sisfo_toko.
2. Jalankan perintah berikut:
3. php spark make:controller Dashboard
4. Buka file app/Controllers/Dashboard.php. Ubah kodenya menjadi seperti berikut:

```
<?php

namespace App\Controllers;

use App\Controllers\BaseController;
use App\Models\ProdukModel;

class Dashboard extends BaseController
{
    public function index()
    {
        $produkModel = new ProdukModel();

        // Menghitung total macam/item produk
        $totalProduk = $produkModel->countAll();

        // Melakukan query manual (Query Builder) untuk menjumlahkan total stok dan nilai aset
        $db = \Config\Database::connect();
        $query = $db->query("SELECT SUM(stok) as total_stok, SUM(harga * stok) as total_aset FROM produk");
        $row = $query->getRow();

        $data = [
```

```
            'title'      => 'Dashboard Toko',
            'total_produk'=> $totalProduk,
            'total_stok'  => $row->total_stok ?? 0,
            'total_aset'  => $row->total_aset ?? 0
        ];

        return view('dashboard/index', $data);
    }
}
```

8.2 Membuat View Dashboard

1. Buat folder baru bernama dashboard di dalam app/Views/.
2. Buat file index.php di dalam app/Views/dashboard/, lalu isi dengan kode berikut:

```
<?= $this->extend('layout/template') ?>
<?= $this->section('content') ?>

<div class="row">
    <div class="col-12">
        <h2 class="mb-4">Dashboard Sistem Informasi Toko</h2>
    </div>
</div>

<div class="row">
    <!-- Card Total Produk -->
    <div class="col-md-4 mb-4">
        <div class="card text-white bg-primary shadow">
            <div class="card-body">
                <h5 class="card-title">Total Item Produk</h5>
                <h2 class="display-5"><?= $total_produk; ?></h2>
                <p class="card-text">Macam barang yang terdaftar.</p>
            </div>
            <div class="card-footer bg-transparent border-white">
                <a href="/produk" class="text-white text-decoration-none">Lihat Detail ➔</a>
            </div>
        </div>
    </div>

    <!-- Card Total Stok -->
    <div class="col-md-4 mb-4">
        <div class="card text-white bg-success shadow">
```

```
            <div class="card-body">
                <h5 class="card-title">Total Stok Barang</h5>
                <h2 class="display-5"><?= $total_stok; ?></h2>
                <p class="card-text">Jumlah fisik barang di gudang.</p>
            </div>
            <div class="card-footer bg-transparent border-white">
                <a href="/laporan" class="text-white text-decoration-none">Lihat Laporan
➔</a>
            </div>
        </div>
    </div>

    <!-- Card Total Aset -->
    <div class="col-md-4 mb-4">
        <div class="card text-white bg-info shadow">
            <div class="card-body">
                <h5 class="card-title">Estimasi Nilai Aset</h5>
                <h2 class="display-6">Rp <?= number_format($total_aset, 0, ',', '.'); ?></h2>
                <p class="card-text">Total (Harga x Stok) seluruh barang.</p>
            </div>
            <div class="card-footer bg-transparent border-white">
                <small>Berdasarkan harga jual saat ini.</small>
            </div>
        </div>
    </div>
</div>

<?= $this->endSection() ?>
```

Bab 9: Membuat Fitur Cetak Laporan

Sistem kasir/toko harus memiliki fitur untuk mencetak laporan. Kita akan membuat fitur cetak laporan stok produk ke format cetak bawaan browser (bisa di-Save as PDF).

9.1 Membuat Controller Laporan

1. Di terminal, jalankan:
2. php spark make:controller Laporan

3. Buka app/Controllers/Laporan.php. Isi dengan kode berikut:

```
<?php

namespace App\Controllers;
```

```
use App\Controllers\BaseController;
use App\Models\ProdukModel;

class Laporan extends BaseController
{
    protected $produkModel;

    public function __construct()
    {
        $this->produkModel = new ProdukModel();
    }

    // Menampilkan halaman data laporan sebelum dicetak
    public function index()
    {
        $data = [
            'title' => 'Laporan Stok Produk',
            'produk' => $this->produkModel->findAll()
        ];
        return view('laporan/index', $data);
    }

    // Menampilkan format halaman cetak (tanpa navbar/layout utama)
    public function cetak_produk()
    {
        $data = [
            'title' => 'Cetak Laporan Produk',
            'produk' => $this->produkModel->findAll()
        ];
        return view('laporan/cetak_produk', $data);
    }
}
```

9.2 Membuat View Laporan (Preview & Print)

1. Buat folder laporan di dalam app/Views/.
2. Buat file **index.php** di dalam app/Views/laporan/ (Ini halaman pratinjau sebelum cetak):

```
<?= $this->extend('layout/template') ?>
<?= $this->section('content') ?>
```

```
<div class="row">
    <div class="col">
        <h2 class="mb-3">Laporan Stok Produk</h2>

        <!-- Tombol Cetak yang mengarah ke tab baru -->
        <a href="/laporan/cetak_produk" target="_blank" class="btn btn-secondary mb-3">
            Cetak Laporan (Print / PDF)
        </a>

        <table class="table table-bordered table-hover">
            <thead class="table-dark">
                <tr>
                    <th>No</th>
                    <th>Kode Produk</th>
                    <th>Nama Produk</th>
                    <th>Harga</th>
                    <th>Stok</th>
                    <th>Nilai Aset (Harga x Stok)</th>
                </tr>
            </thead>
            <tbody>
                <?php
                $i = 1;
                $grand_total = 0;
                foreach($produk as $p) :
                    $subtotal = $p['harga'] * $p['stok'];
                    $grand_total += $subtotal;
                ?>
                <tr>
                    <td><?= $i++; ?></td>
                    <td><?= $p['kode_produk']; ?></td>
                    <td><?= $p['nama_produk']; ?></td>
                    <td>Rp <?= number_format($p['harga'], 0, ',', '.'); ?></td>
                    <td><?= $p['stok']; ?></td>
                    <td>Rp <?= number_format($subtotal, 0, ',', '.'); ?></td>
                </tr>
                <?php endforeach; ?>
            </tbody>
            <tfoot>
```

```
            <tr class="table-active fw-bold">
                <td colspan="5" class="text-end">Total Nilai Aset Keseluruhan:</td>
                <td>Rp <?= number_format($grand_total, 0, ',', '.'); ?></td>
            </tr>
        </tfoot>
    </table>
  </div>
</div>

<?= $this->endSection() ?>
```

3. Buat file **cetak_produk.php** di dalam app/Views/laporan/. *Catatan: File ini tidak menggunakan layout/template bawaan karena kita ingin halamannya bersih warna putih layaknya kertas.*

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?= $title; ?></title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    .text-center { text-align: center; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #000; padding: 8px; text-align: left; }
    th { background-color: #f2f2f2; }
    .text-right { text-align: right; }
  </style>
</head>
<body>
  <div class="text-center">
    <h2>SISTEM INFORMASI TOKO SMK</h2>
    <p>Jl. Pendidikan No. 1, Kota Belajar</p>
    <hr>
    <h3>LAPORAN STOK DAN ASET PRODUK</h3>
    <p>Tanggal Cetak: <?= date('d M Y H:i:s'); ?></p>
  </div>

  <table>
    <thead>
```

```
        <tr>
            <th>No</th>
            <th>Kode Produk</th>
            <th>Nama Produk</th>
            <th>Harga Satuan</th>
            <th>Stok</th>
            <th>Subtotal Aset</th>
        </tr>
    </thead>
    <tbody>
        <?php
        $i = 1;
        $grand_total = 0;
        foreach($produk as $p) :
            $subtotal = $p['harga'] * $p['stok'];
            $grand_total += $subtotal;
        ?>
        <tr>
            <td><?= $i++; ?></td>
            <td><?= $p['kode_produk']; ?></td>
            <td><?= $p['nama_produk']; ?></td>
            <td>Rp <?= number_format($p['harga'], 0, ',', '.'); ?></td>
            <td><?= $p['stok']; ?></td>
            <td class="text-right">Rp <?= number_format($subtotal, 0, ',', '.'); ?></td>
        </tr>
        <?php endforeach; ?>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="5" class="text-right">TOTAL NILAI ASET:</th>
            <th class="text-right">Rp <?= number_format($grand_total, 0, ',', '.'); ?></th>
        </tr>
    </tfoot>
</table>

<br><br>
<div style="float: right; text-align: center;">
    <p>Mengetahui,</p>
    <br><br><br>
    <p><strong>Kepala Toko / Admin</strong></p>
```

```
    </div>

    <!-- Script otomatis memunculkan dialog print -->
    <script>
        window.print();
    </script>
</body>
</html>
```

Bab 10: Menggabungkan Sistem (Navigasi & Routing)

Agar Dashboard, Produk, dan Laporan saling terhubung, kita perlu memperbarui Menu Navigasi (Navbar) dan sistem Routing (Rute URL).

10.1 Memperbarui Navbar (Template)

Buka file app/Views/layout/template.php. Cari bagian <nav> dan ganti bagian menu <ul> menjadi seperti ini:

```
<!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
    <div class="container">
        <a class="navbar-brand" href="/">Toko SMK</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a class="nav-link" href="/">Dashboard</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="/produk">Kelola Produk</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="/laporan">Laporan</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

(Tips: Saya mengganti bg-primary menjadi bg-dark agar terlihat lebih elegan seperti sistem admin).

10.2 Memperbarui Routes (Penting)

Buka file app/Config/Routes.php. Sesuaikan rute aplikasinya menjadi seperti ini:

```
use CodeIgniter\Router\RouteCollection;

/**
 * @var RouteCollection $routes
 */
// Jadikan Dashboard sebagai halaman utama (Root)
$routes->get('/', 'Dashboard::index');

// Routes untuk Produk (CRUD)
$routes->get('/produk', 'Produk::index');
$routes->get('/produk/create', 'Produk::create');
$routes->post('/produk/save', 'Produk::save');
$routes->get('/produk/edit/(:num)', 'Produk::edit/$1');
$routes->post('/produk/update/(:num)', 'Produk::update/$1');
$routes->delete('/produk/(:num)', 'Produk::delete/$1');

// Routes untuk Laporan
$routes->get('/laporan', 'Laporan::index');
$routes->get('/laporan/cetak_produk', 'Laporan::cetak_produk');
```

Bab 11: Uji Coba Akhir

Selamat! Aplikasi Anda sekarang sudah utuh. Mari kita uji coba:

1. Jalankan kembali server (jika mati) menggunakan php spark serve.
2. Buka http://localhost:8080/. Anda akan disambut oleh halaman **Dashboard** dengan kartu statistik berwarna yang otomatis menghitung total produk dan aset.
3. Klik menu **Laporan** di navigasi atas.
4. Klik tombol abu-abu **"🖨 Cetak Laporan (Print / PDF)"**.
5. Browser akan membuka tab baru berlatar putih dengan format laporan resmi, dan **jendela dialog Print komputer Anda akan otomatis terbuka** (Bisa Anda pilih *Save as PDF* atau cetak dengan printer sungguhan).

Sistem Informasi Toko berbasis MVC dengan framework modern CodeIgniter 4 Anda telah sempurna! 🎉

Lanjutan Modul CodeIgniter 4: Fitur Transaksi Penjualan

Fitur transaksi adalah inti dari aplikasi Point of Sales (POS). Pada bab ini, kita akan membuat fitur kasir sederhana. Logikanya: Kasir memilih produk dan memasukkan jumlah beli -> Sistem menghitung total harga -> Data transaksi disimpan -> **Stok produk otomatis berkurang**.

Bab 12: Membuat Fitur Transaksi Penjualan

12.1 Membuat Tabel Transaksi (Migration)

Pertama, kita butuh tabel untuk menyimpan riwayat penjualan. Kita akan menggunakan fitur Migration lagi.

1. Buka terminal di VS Code, jalankan perintah:
2. `php spark make:migration Transaksi`
3. Buka file migrasi baru di folder `app/Database/Migrations/` (namanya berakhiran `_Transaksi.php`). Ubah kodenya menjadi:

```
<?php

namespace App\Database\Migrations;

use CodeIgniter\Database\Migration;

class Transaksi extends Migration
{
    public function up()
    {
        $this->forge->addField([
            'id_transaksi' => [
                'type'           => 'INT',
                'constraint'     => 11,
                'unsigned'       => true,
                'auto_increment' => true,
            ],
            // id_produk digunakan untuk relasi ke tabel produk
            'id_produk' => [
                'type'           => 'INT',
                'constraint'     => 11,
                'unsigned'       => true,
            ],
            'jumlah' => [
                'type'       => 'INT',
                'constraint' => 11,
            ],
            'total_harga' => [
```

```
                'type'       => 'INT',
                'constraint' => 11,
            ],
            'created_at' => [
                'type' => 'DATETIME',
                'null' => true,
            ],
            'updated_at' => [
                'type' => 'DATETIME',
                'null' => true,
            ],
        ]);
        $this->forge->addKey('id_transaksi', true);
        $this->forge->createTable('transaksi');
    }

    public function down()
    {
        $this->forge->dropTable('transaksi');
    }
}
```

53. Jalankan migrasi di terminal untuk mengeksekusinya ke database:

```
php spark migrate
```

12.2 Membuat Model Transaksi

Kita perlu Model untuk tabel `transaksi`. Selain itu, kita akan membuat fungsi khusus (Query Builder) untuk menggabungkan (JOIN) tabel `transaksi` dan `produk` agar kita bisa menampilkan nama produk, bukan sekadar ID-nya.

1. Di terminal, jalankan:

```
php spark make:model TransaksiModel
```

3. Buka `app/Models/TransaksiModel.php` dan sesuaikan isinya:

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class TransaksiModel extends Model
{
    protected $table            = 'transaksi';
    protected $primaryKey       = 'id_transaksi';
    protected $useAutoIncrement = true;
```

```
    protected $returnType       = 'array';

    protected $allowedFields    = ['id_produk', 'jumlah', 'total_harga'];

    protected $useTimestamps = true;
    protected $createdField  = 'created_at';
    protected $updatedField  = 'updated_at';

    // Fungsi untuk mengambil data transaksi beserta nama produk (JOIN)
    public function getTransaksiLengkap()
    {
        return $this->db->table('transaksi')
            ->join('produk', 'produk.id_produk = transaksi.id_produk')
            ->orderBy('transaksi.created_at', 'DESC')
            ->get()->getResultArray();
    }
}
```

12.3 Membuat Controller Transaksi

Controller ini akan menangani dua hal: Menampilkan daftar riwayat transaksi, dan memproses form kasir (mengurangi stok).

1. Di terminal, jalankan:

```
php spark make:controller Transaksi
```

3. Buka `app/Controllers/Transaksi.php` dan ubah menjadi seperti ini:

```
<?php

namespace App\Controllers;

use App\Controllers\BaseController;
use App\Models\TransaksiModel;
use App\Models\ProdukModel;

class Transaksi extends BaseController
{
    protected $transaksiModel;
    protected $produkModel;

    public function __construct()
    {
```

```
19.         $this->transaksiModel = new TransaksiModel();
20.         $this->produkModel = new ProdukModel();
21.     }
22.
23.     // Menampilkan riwayat transaksi
24.     public function index()
25.     {
26.         $data = [
27.             'title' => 'Riwayat Transaksi',
28.             'transaksi' => $this->transaksiModel-
  >getTransaksiLengkap()
29.         ];
30.         return view('transaksi/index', $data);
31.     }
32.
33.     // Menampilkan form kasir
34.     public function create()
35.     {
36.         $data = [
37.             'title' => 'Kasir - Transaksi Baru',
38.             // Mengambil data produk yang stoknya lebih
  dari 0 untuk dipilih di dropdown
39.             'produk' => $this->produkModel->where('stok
  >', 0)->findAll()
40.         ];
41.         return view('transaksi/create', $data);
42.     }
43.
44.     // Memproses penyimpanan transaksi dan potong stok
45.     public function save()
46.     {
47.         $id_produk = $this->request->getVar('id_produk');
48.         $jumlah_beli = $this->request->getVar('jumlah');
49.
50.         // 1. Cari tahu harga dan stok produk saat ini
51.         $produk = $this->produkModel->find($id_produk);
52.
53.         // 2. Cek apakah stok mencukupi
54.         if ($produk['stok'] < $jumlah_beli) {
55.             session()->setFlashdata('error', 'Stok produk
  tidak mencukupi!');
```

```
            return redirect()->back();
        }

        // 3. Hitung total harga otomatis
        $total_harga = $produk['harga'] * $jumlah_beli;

        // 4. Simpan ke tabel transaksi
        $this->transaksiModel->save([
            'id_produk'   => $id_produk,
            'jumlah'      => $jumlah_beli,
            'total_harga' => $total_harga
        ]);

        // 5. Kurangi stok di tabel produk (Update Stok)
        $sisa_stok = $produk['stok'] - $jumlah_beli;
        $this->produkModel->save([
            'id_produk' => $id_produk,
            'stok'      => $sisa_stok
        ]);

        session()->setFlashdata('pesan', 'Transaksi
  berhasil disimpan. Stok telah dikurangi.');
        return redirect()->to('/transaksi');
    }
}
```

12.4 Membuat View Transaksi

Buat folder baru bernama `transaksi` di dalam folder `app/Views/`.

1. Halaman Index (Riwayat Transaksi) Buat file `index.php` di dalam `app/Views/transaksi/`:

```
<?= $this->extend('layout/template') ?>
<?= $this->section('content') ?>

<div class="row">
    <div class="col">
        <h2 class="mb-3">Riwayat Transaksi</h2>

        <?php if(session()->getFlashdata('pesan')) : ?>
            <div class="alert alert-success" role="alert">
                <?= session()->getFlashdata('pesan'); ?>
            </div>
        <?php endif; ?>
```

```
        <a href="/transaksi/create" class="btn btn-success mb-
3">+ Buat Transaksi Baru</a>

        <table class="table table-bordered table-striped">
            <thead class="table-dark">
                <tr>
                    <th>No</th>
                    <th>Waktu Transaksi</th>
                    <th>Nama Produk</th>
                    <th>Jumlah Beli</th>
                    <th>Total Harga</th>
                </tr>
            </thead>
            <tbody>
                <?php $i = 1; foreach($transaksi as $t) : ?>
                <tr>
                    <td><?= $i++; ?></td>
                    <td><?= date('d-m-Y H:i',
strtotime($t['created_at'])); ?></td>
                    <td><?= $t['nama_produk']; ?></td>
                    <td><?= $t['jumlah']; ?></td>
                    <td>Rp <?= number_format($t['total_harga'],
0, ',', '.'); ?></td>
                </tr>
                <?php endforeach; ?>
                <?php if(empty($transaksi)) : ?>
                    <tr>
                        <td colspan="5" class="text-
center">Belum ada transaksi.</td>
                    </tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<?= $this->endSection() ?>
```

2. Halaman Create (Form Kasir) Buat file `create.php` di dalam `app/Views/transaksi/`:

```
<?= $this->extend('layout/template') ?>
```

```
<?= $this->section('content') ?>

<div class="row">
    <div class="col-8">
        <h2 class="my-3">Kasir - Transaksi Baru</h2>

        <?php if(session()->getFlashdata('error')) : ?>
            <div class="alert alert-danger" role="alert">
                <?= session()->getFlashdata('error'); ?>
            </div>
        <?php endif; ?>

        <div class="card shadow mb-4">
            <div class="card-body">
                <form action="/transaksi/save" method="post">
                    <?= csrf_field(); ?>

                    <div class="mb-3">
                        <label for="id_produk" class="form-
label">Pilih Produk</label>
                        <select class="form-select"
id="id_produk" name="id_produk" required>
                            <option value="">-- Pilih Produk --
</option>
                            <?php foreach($produk as $p) : ?>
                                <option value="<?=
$p['id_produk']; ?>">
                                    <?= $p['nama_produk']; ?>
(Stok: <?= $p['stok']; ?> | Harga: Rp <?=
number_format($p['harga'], 0, ',', '.'); ?>)
                                </option>
                            <?php endforeach; ?>
                        </select>
                    </div>

                    <div class="mb-3">
                        <label for="jumlah" class="form-
label">Jumlah Beli</label>
                        <input type="number" class="form-
control" id="jumlah" name="jumlah" min="1" required>
                    </div>
```

```
                    <button type="submit" class="btn btn-
success">Proses Pembayaran</button>
                    <a href="/transaksi" class="btn btn-
secondary">Kembali</a>
                </form>
            </div>
        </div>

    </div>
</div>

<?= $this->endSection() ?>
```

12.5 Memperbarui Navbar dan Routes

Terakhir, mari kita daftarkan menu dan URL-nya agar bisa diakses.

1. Update Navbar (`app/Views/layout/template.php`) Cari blok `<ul>` pada bagian navbar, lalu tambahkan menu Transaksi di antara menu Produk dan Laporan:

```
<ul class="navbar-nav me-auto">
    <li class="nav-item">
        <a class="nav-link" href="/">Dashboard</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" href="/produk">Kelola Produk</a>
    </li>
    <!-- Tambahkan baris ini -->
    <li class="nav-item">
        <a class="nav-link text-warning fw-bold"
href="/transaksi">Kasir / Transaksi</a>
    </li>
    <!-- ================== -->
    <li class="nav-item">
        <a class="nav-link" href="/laporan">Laporan</a>
    </li>
</ul>
```

2. Update Routes (`app/Config/Routes.php`) Tambahkan *routes* berikut di bagian bawah file:

```
// Routes untuk Transaksi
$routes->get('/transaksi', 'Transaksi::index');
$routes->get('/transaksi/create', 'Transaksi::create');
$routes->post('/transaksi/save', 'Transaksi::save');
```

Uji Coba Fitur Transaksi!

1. Jalankan `php spark serve`.
2. Buka browser. Coba klik menu **Kasir / Transaksi**.
3. Klik tombol **Buat Transaksi Baru**.
4. Pilih produk dari *dropdown* (hanya produk yang ada stoknya yang akan muncul) dan masukkan jumlah belinya.
5. Klik **Proses Pembayaran**.
6. **Momen Pembuktian:**
    - Cek daftar transaksi, pastikan Total Harga terhitung benar secara otomatis.
    - Cek menu **Kelola Produk** atau **Dashboard**, pastikan stok produk yang Anda beli tadi sudah berkurang secara otomatis!

Selamat! Aplikasi Anda sekarang sudah benar-benar hidup layaknya sebuah aplikasi mesin Kasir / POS sungguhan. 🎉